Final Defense Script Aligned with the Current PPT

This script is fully aligned with the current presentation file:

- `D:\杞⍰欢娴嫻瘓\final_presentation_ppt.pdf`

It is written so that the team can directly use it on stage without changing the slides.

The presentation currently has `10` slides, and this script follows the same order and emphasis.

Speaking Plan

- `Yi Xu`: Slides 1-2
- `Member 3`: Slides 3-4
- `Member 4`: Slides 5-6
- `Member 5`: Slides 7-8
- `Luowu Zhang`: Slides 9-10

If only one representative presents, this script can still be used continuously from Slide 1 to Slide 10.

Slide 1. Title

Speaker: Yi Xu
Suggested time: 35-45 seconds

Speaker script

Good morning, everyone. We are Group 7, and our project is called ARG-Test: Auditable Requirement-Driven Black-Box Test Generation with Structured LLM Reasoning and Contract Verification.

Our project focuses on using AI to support black-box testing from natural-language requirements. Instead of directly asking an LLM to generate a list of test cases, we designed a structured and auditable pipeline that produces more reliable testing artifacts.

Slide 2. Problem and Scope

Speaker: Yi Xu
Suggested time: 55-70 seconds

Speaker script

This slide summarizes what we did and defines the scope of our project.

The field of our work is AI-enhanced black-box dynamic testing. The input to our system is natural-language system requirements, and the output is a set of auditable and structured test cases.

Our core objective is the following: given natural-language requirements, we generate black-box test cases with structured reasoning and verification.

In other words, our goal is not just to make the model produce test cases, but to make the whole generation process more transparent, easier to inspect, and easier to improve.

Slide 3. Requirement Analysis and Input

Speaker: Member 3
Suggested time: 55-70 seconds

Speaker script

Our testing scenarios mainly come from e-commerce and business-platform rules. We intentionally kept the dataset within a consistent application domain so that the experimental setting is coherent and the resulting test cases are easier to analyze.

For the dataset split, we used 50 requirements in the development set and 16 requirements in the test set.

These requirements cover three main categories: business rules, input validation, and workflow state transitions.

This design allows us to test whether the method works across different types of black-box testing requirements rather than only one narrow scenario.

Slide 4. Tool Artifact and Prompt Design

Speaker: Member 3
Suggested time: 60-75 seconds

Speaker script

For the formal experiments, our core model was Qwen3.5-Flash.

We did not use only one prompt. Instead, we adopted a prompt family approach.

The system prompt defines the overall behavior of the testing assistant. The generation prompt asks the model to produce a structured reasoning trace. The repair prompt is used to fix weak or failed outputs based on checker feedback. In addition, we also keep a plain baseline prompt for comparison.

This design is important because it reflects team-AI interaction. The final result does not come from one single model call. It comes from a guided prompting strategy designed and revised by the team.

Slide 5. Pipeline and Architecture

Speaker: Member 4
Suggested time: 60-75 seconds

Speaker script

This slide shows the complete pipeline workflow of ARG-Test.

The pipeline contains six major stages. First, we generate a structured trace from the requirement. Second, we parse the model output into a typed representation. Third, we run a checker suite for contract verification. Fourth, we rerank multiple candidates. Fifth, we apply a repair loop when necessary. Finally, we export the result as the final testing artifact.

So our tool is not just prompt in and answer out. It is a complete pipeline that combines generation, checking, selection, and refinement.

Slide 6. Generated Output Example

Speaker: Member 4
Suggested time: 50-65 seconds

Speaker script

This slide explains the format of our generated output.

Our outputs are structured test suites rather than free-form text. Each output explicitly contains fields such as the testing technique, the concrete input, the expected output, and the covered requirement item.

This structure makes the output easier to inspect and compare. It also makes evaluation possible, because we can analyze whether the generated suite really covers the important aspects of the requirement instead of only sounding reasonable in natural language.

Slide 7. Main Result and Baselines

Speaker: Member 5
Suggested time: 70-85 seconds

Speaker script

This slide presents the main experimental results and the baseline comparison.

For our full pipeline, the average checker score is 0.959, and the average overall coverage is 0.615.

We also compare our approach against several baselines. The rule-based baseline reaches a coverage of 0.147. The plain LLM baseline reaches only 0.030. The structured version without checker

reaches 0.538.

These results show that our final pipeline clearly outperforms both the traditional non-AI baseline and the plain prompt-only baseline. They also show that structured generation alone already helps, but the full pipeline performs best overall.

Slide 8. Verification, Ablation, and Generalization

Speaker: Member 5
Suggested time: 70-85 seconds

Speaker script

This slide explains why the verification components matter and how well the method generalizes.

From the ablation perspective, the checker-repair stage significantly improves checker score and overall reliability. This means the generated test cases are more auditable and more consistent with the intended testing logic.

From the generalization perspective, we report category-level coverage on the test set. For business rules, the coverage is 0.577. For input validation, it is 0.579. For workflow-state requirements, it is 0.697.

These numbers show that the method is not restricted to one type of requirement. It works across several common black-box testing scenarios.

Slide 9. Team-AI Interaction and Revision

Speaker: Luowu Zhang
Suggested time: 70-85 seconds

Speaker script

This slide highlights the interaction between the team and AI, and how we revised the system step by step.

In the early version, the model often produced unstructured free-form text. It also sometimes hallucinated rules or missed important testing obligations. In some cases, the generated test cases were weak or failed to satisfy the expected testing logic.

To address these issues, we introduced structured reasoning traces. Then we added parsing and contract checkers. After that, we implemented reranking and repair loops.

So the final system is the result of an iterative revision process. The team did not simply accept the model's first answer. Instead, we continuously analyzed the weaknesses of early outputs and refined the workflow until it became a stable testing artifact.

Slide 10. Conclusion

Speaker: Luowu Zhang
Suggested time: 50-65 seconds

Speaker script

To conclude, ARG-Test shows that structured LLM reasoning plus contract verification can produce more auditable and higher-coverage testing artifacts.

Our method outperforms both prompt-only baselines and traditional rule-based methods, which means the project successfully demonstrates the value of combining AI generation with explicit verification and refinement.

At the same time, we still see room for future work. We can extend the input branch from natural-language requirements to source codebases, and we can further automate the execution of generated scripts.

Overall, we believe this project satisfies the assignment requirements and demonstrates a complete AI-enhanced testing workflow.

One-Presenter Transition Version

If one person presents all 10 slides, use these transitions:

- Slide 1 to 2: `After introducing the project, I will now explain the testing problem and our scope.`
- Slide 2 to 3: `Next, I will show how our requirement inputs and dataset are organized.`
- Slide 3 to 4: `Based on these inputs, we designed a prompt-based tool artifact with several prompt roles.`
- Slide 4 to 5: `This leads directly to our full pipeline and architecture.`
- Slide 5 to 6: `After the pipeline, I will briefly explain the structure of the generated outputs.`
- Slide 6 to 7: `Then I will move to the main results and compare our method with baselines.`
- Slide 7 to 8: `Next, I will explain our verification, ablation, and generalization findings.`
- Slide 8 to 9: `After that, I will describe how the team iteratively refined the system with AI.`
- Slide 9 to 10: `Finally, I will conclude with the key takeaway and future work.`

Short Q&A Backup

Q1. Why did you choose requirements as the input?

Because requirement-driven black-box testing is fully allowed by the assignment, and it is a strong fit for structured reasoning and coverage-oriented evaluation.

Q2. Why is your method better than plain prompting?

Because we do not rely on a single prompt output. We use structured reasoning, parsing, checking, reranking, and repair to improve reliability and auditability.

Q3. Does the checker prove that the test cases are fully correct?

No. The checker does not provide formal proof. It verifies whether the generated reasoning and test suite satisfy important testing obligations and contracts.

Q4. What is the main contribution of the team?

The team designed the prompts, the pipeline, the checking logic, the dataset, the experiment setup, and the final revision strategy. AI is one part of the workflow, not the whole workflow by itself.

Q5. Where is the tool? Is this really a tool, or just a testing idea?

Yes, it is a real tool rather than only a testing idea. The tool is the ARG-Test pipeline itself. It takes requirement files as input, calls the model with structured prompts, parses the responses, checks them with contract-based validators, reranks candidates, optionally repairs weak outputs, and finally exports structured test suites and evaluation reports. So the tool is the whole runnable workflow, not just the final experiment or a single prompt.

Q6. What are the input and output of your tool?

The main input of our current implementation is a natural-language requirement. The output is a structured black-box test suite together with intermediate and final artifacts, such as reasoning traces, checker results, coverage reports, and exported test cases. So the system does not only generate text; it generates test artifacts that can be inspected and evaluated.

Q7. How do you define a test case in your project?

In our project, a test case is defined as a structured testing unit derived from one or more requirement items. At minimum, it contains a concrete input or scenario, the expected output or expected system behavior, and the requirement item or testing obligation it is supposed to cover. In many cases, we also record the testing technique, such as equivalence partitioning or boundary value analysis, to make the case easier to audit.

Q8. Based on what do you define or derive the test cases?

We define and derive test cases based on classic black-box testing principles and the requirement constraints themselves. In particular, we use equivalence partitioning, boundary value analysis,

decision-rule reasoning, and workflow or state-transition reasoning when appropriate. So the model is not generating cases arbitrarily. The cases are guided by testing theory, requirement rules, and our checker contracts.

Q9. Why did you not implement the codebase-input branch?

The assignment allows either system requirements or testing codebase as the starting point. We chose the requirement-driven branch because it is a cleaner fit for black-box testing and for the structured reasoning idea from the reference paper. We therefore focused on doing one branch well rather than doing two branches superficially. In future work, the same framework can be extended to codebase input by adding a code-to-requirement or code-to-contract adaptation layer.

Q10. Why did you use Qwen3.5-Flash? Is it reliable enough?

We selected Qwen3.5-Flash because it offered a practical balance between cost, speed, and output quality for a course project. More importantly, our project does not rely on trusting the raw model output directly. We wrap the model inside a structured pipeline with parsing, checking, reranking, and repair. So the reliability of the final artifact comes from the whole system design, not from assuming the base model is always correct by itself.

Q11. How do you know your checker is valid?

Our checker is not intended to be a full semantic proof engine. Its role is narrower and more practical: it verifies whether the generated reasoning trace and final test suite satisfy explicit testing obligations. These obligations come from standard black-box testing logic, such as covering valid and invalid partitions, boundary points, decision rules, and workflow transitions. So the checker is valid as a contract-based verification layer, although it does not guarantee complete real-world correctness.

Q12. Your average coverage is 0.615. Why do you still claim success?

We claim success because the goal of the project is not to achieve perfect coverage on every requirement. The goal is to build an AI-enhanced testing tool that is structured, auditable, and clearly better than simpler baselines. In that sense, the project is successful because the full pipeline substantially outperforms the rule-based and plain-prompt baselines, while also producing high checker scores and stable structured artifacts. So success here means meaningful improvement and a complete working tool, not perfect coverage.

Q13. Is the comparison with baselines fair?

Yes, the comparison is fair in the sense that all baselines solve the same requirement-to-test generation task on the same dataset. The rule-based baseline represents a traditional non-AI approach. The plain LLM baseline represents direct prompting without structural control. The structured-no-checker baseline isolates the effect of structure alone. The full pipeline then shows the added value of checking, reranking, and repair. This design allows us to compare not only final performance, but also the contribution of each component.

Q14. What is the biggest limitation of your current project?

The biggest limitation is that our current implementation focuses on requirement-driven testing and on structured generation quality, rather than full executable end-to-end validation against a large real software system. In addition, our dataset size is still course-project scale rather than industrial scale. So our conclusions should be interpreted as strong evidence that the method is effective in this setting, not as a claim of universal testing completeness.

Q15. If the model output is wrong, what does your system do?

If the raw model output is weak, inconsistent, or incomplete, our system does not directly accept it. Instead, it parses the output, checks whether important testing obligations are missing, compares candidates, and may run a repair stage. This means the system has a built-in mechanism to detect and mitigate weak generations rather than fully trusting the first response.